GRADIENT DESCENT + LINEAR REGRESSION

M observations, N features; $x_{io} = 1$. **Model:** $z_i = w^T x_i = w_o + \sum_{j=1}^{N} w_j x_{ij}$. Training minimizes J over w; x, y are fixed.
$J = MSE = (1/M) \sum_i (y_i - z_i)^2$
$\partial J/\partial w_j = -(2/M) \sum_i (y_i - z_i) x_{ij}$
$\nabla J = (2/M) X^T(Xw - y)$
Why −2 and x_{ij}? Chain rule: derivative of the square gives $2(y_i - z_i)$; the inside derivative is $-x_{ij}$. For w_o, $x_{io} = 1$.
GD: $w_{k+1} = w_k - \eta \nabla J(w_k)$
Initialize w → gradient → update **all coordinates using old w** → repeat until $\|w_{k+1} - w_k\|_2 <$ tol (or iteration limit).
$-\nabla J$ = steepest local decrease; ∇J points uphill. Its norm is **not** distance to the optimum; it need not point straight at it.
η too big: overshoot/diverge; **too small:** slow. Adaptive example: $\eta_k = 1/k$. **tol↓:** more accurate/more steps; **tol↑:** faster/less accurate. Course: η, tol are **algorithm settings, not model hyperparameters**; no CV.
Scale: unequal feature magnitudes → elongated bowl → zig-zag/slow GD. Scaling improves conditioning, not convexity.
Convex: $\nabla^2 J = (2/M) X^T X \geq 0$. Full column rank → strictly convex bowl, unique global minimum.
Least squares: $\hat{w} = (X^T X)^{-1} X^T y$
Closed form if invertible; costly at scale. GD is numerical.

REGULARIZATION: FIT + PENALTY

Ridge: $J = MSE + \lambda \|w\|_2^2$
Lasso: $J = MSE + \lambda \|w\|_1$
ElasticNet: $J = MSE + \lambda_1 \|w\|_1 + \lambda_2 \|w\|_2^2$
$\|w\|_1 = \sum_j |w_j|$; $\|w\|_2^2 = \sum_j w_j^2$. **λ↑:** more shrinkage/regularization, less overfit; too large → underfit. λ is a **model hyperparameter**.
Ridge: smooth; closed form or plain GD. $\nabla J = \nabla MSE + 2\lambda w$.
Lasso: sets some weights **exactly to 0** (feature selection). |w| is nondifferentiable at 0 → no plain GD/general closed form; use subgradient/coordinate descent. ElasticNet has the same issue.
ElasticNet: $\lambda_1 = 0$ gives Ridge; $\lambda_2 = 0$ gives Lasso. Ideal tuning including endpoints can match either's best selection score; tuning two parameters costs more. Neither Ridge nor Lasso always wins.
Slides penalize w_o. If the intercept is excluded, omit its penalty and penalty-gradient term.

LOGISTIC REGRESSION + DERIVATIONS

$p_i = P(y_i = 1 \mid x_i) = \sigma(z_i) = 1/(1 + e^{-z_i})$
$\sigma'(z) = e^{-z}/(1 + e^{-z})^2 = \sigma(z)[1 - \sigma(z)]$
Split: $[1/(1 + e^{-z})] \cdot [e^{-z}/(1 + e^{-z})]$. Factor 1 = σ(z); factor 2 = $[(1 + e^{-z}) - 1]/(1 + e^{-z}) = 1 - \sigma(z)$.
$J = BCE = -(1/M) \sum_i [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)]$
y = 1: loss $-\ln p$. **y = 0:** loss $-\ln(1 - p)$. Confident correct → 0; confident wrong → ∞. Independent Bernoulli likelihood: $L = \prod_i p_i^{y_i}(1 - p_i)^{1-y_i}$; BCE = −ln L/M.
$\partial \ell_i/\partial p_i = (p_i - y_i)/[p_i(1 - p_i)]$
$\partial p_i/\partial w_j = p_i(1 - p_i) x_{ij}$
$\Rightarrow \partial J/\partial w_j = (1/M) \sum_i (p_i - y_i) x_{ij}$
$\Rightarrow \nabla J = X^T(p - y)/M$
Why BCE? Convex in w; sigmoid-MSE is nonconvex. No closed form → GD in class. Regularized classification = BCE + the same penalties.
Predict 1 if $p \geq$ threshold ($0.5 \Leftrightarrow z \geq 0$). Multi-class: categorical cross-entropy. Perfect separation: unregularized weights can diverge; **no finite max-margin convergence**.

HIERARCHY + LARGE DATA

Agglomerative: singletons → merge. **Divisive:** one cluster → split. Dendrogram height = distance; a horizontal cut sets clusters. Good for small data; costly/messy at scale. Mini-batch K-Means/BIRCH help large data.

SUPPORT VECTOR MACHINES

Large margin aids generalization. $f = b + v^T x$; v = slopes (**norm excludes b**). Labels: **−1, +1**.
Boundary: f = 0; parallel support planes: $f = \pm a$
Point-to-boundary distance = $|f(x)|/\|v\|_2$
Full margin = $2a/\|v\|_2$; normalize a = 1
Derive: subtract support-plane equations → projected separation × $\|v\|$ = 2a. Max margin ⇔ min $\|v\|$ **subject to no margin violations**. Without constraints: v = 0.
Hard margin: min $\|v\|$; $y_i f(x_i) \geq 1$ for all i
Requires separability. Support vectors touch the margin; soft-margin violations can also be support vectors.
Soft margin: $H = \sum_i \max(0, 1 - y_i z_i) + \lambda \|w\|_2^2$
Score $z_i = w^T x_i$; signed margin $m_i = y_i z_i$. **Hinge:** $h_i = \max(0, 1 - m_i)$.
m ≥ 1: correct, outside/on margin → loss 0.
0 < m < 1: correct but inside margin → 1 − m.
m = 0: on boundary → 1.
m < 0: misclassified → loss > 1.
λ↑: wider margin, more regularization/tolerance of violations; too high → underfit. **λ↓:** narrower margin, less regularization; too low → overfit. **$C \propto 1/\lambda$** acts oppositely.
Convex, nonsmooth at m = 1 → GD extensions (or quadratic programming). Kernel SVM handles nonlinear boundaries; costly for big data. Scale features.
SVC needs extensions for probabilities. Soft margins tolerate outliers; logistic optimizes BCE.
SVR: regression; course: "outermost" support vectors. Zero loss inside ε-tube; penalize observations outside.

MULTI-CLASS: ONE TARGET, C CLASSES

KNN/trees: multiple classes by votes; odd K can still tie (2–2–1). Binary logistic/SVM need extensions.
OvR = OvA: C models, c vs rest; pick largest positive-class score. Several say 1 → most confident positive; all 0 → least confident negative.
OvO: train **$C(C - 1)/2$** models, one per pair using only that pair's data. Majority vote; confidence breaks ties. C = 6 → 15 models.
OvR: linear count, more imbalance (under/oversampling can help). **OvO:** quadratic count, smaller datasets; can be faster for few classes/large data. Neither always wins.

METRICS: READ THE MATRIX AXES FIRST

With **rows = actual, columns = predicted**, for class c:
TP: diagonal cell; **FP:** other entries in its column;
FN: other entries in its row; TN = M − TP − FP − FN.
$P_c = TP/(TP + FP)$ = diagonal / column sum
$R_c = TP/(TP + FN)$ = diagonal / row sum
Macro $P = (1/C)\sum_c P_c$; macro $R = (1/C)\sum_c R_c$
Macro = equal class weights. Bad minority performance hurts; not a support-weighted average.
Micro $P = \sum TP/(\sum TP + \sum FP)$
Micro $R = \sum TP/(\sum TP + \sum FN)$
Micro = pool counts first. Majority dominates, can hide minority failure. Single-label multi-class, all classes: **micro P = micro R = accuracy**.
Accuracy: sum diagonal / M; uniform-guess baseline 1/C. **ROC-AUC:** one OvR curve/AUC per class.

MULTI-LABEL: L TARGETS PER OBSERVATION

Binary relevance: L independent models; ignores target correlations. **Label powerset:** one class per label combination; classification only, up to 2^L classes → impractical.
Classifier/regressor chain: L models, adding previous target predictions to x; uses target correlations. Order matters; errors can propagate.
Hamming loss = $(1/(ML)) \sum_i \sum_l I(y_{il} \neq \hat{y}_{il})$
Wrong labels / all entries. Lower is better; 0 = perfect. For continuous responses, pool entries for RMSE/MAE.
1 − Hamming loss = labelwise accuracy (slides call it "Jaccard"). Standard Jaccard = TP/(TP + FP + FN), comparing positive-label sets.

K-MEANS + LLOYD'S ALGORITHM

No target y. Input **K**; hard labels, no probabilities/noise. Course: Gaussian, diagonal Σ (uncorrelated). Favors round clusters. Scale.
Assignment: $c_i = \arg\min_c \|x_i - \mu_c\|_2$
Update: $\mu_c = (1/n_c) \sum_{i:c_i=c} x_i$
Initialize labels → update first; centroids → assign first. Alternate until stable. Different starts → different **local optima**; restarts/spread-out seeds (K-Means++) help.
Inertia: $I = \sum_i \|x_i - \mu_{c_i}\|_2^2$
$I \geq 0$; optimum nonincreasing as K↑; K = M → 0. **Elbow:** K where decrease slows sharply. Lowest inertia alone overselects K; elbow can be unclear.

K-MEANS EXTENSIONS

K-Modes: categorical features; centroid = featurewise **mode**, assignment by **matching dissimilarity** $d_{cat}(x, u) = \sum_j I(x_j \neq u_j)$, the number of mismatches.
K-Prototypes: mixed numeric + categorical; mean for numeric coordinates, mode for categorical.
$d_{total} = d_{Euclidean,numeric} + \gamma d_{cat}$ [course formula]
γ balances continuous distances vs integer mismatch counts so neither dominates. E.g. [red, large, metal] vs [red, small, wood] → 0 + 1 + 1 = **2** mismatches.
K-Medoids: actual observation minimizing total distance to its cluster's points. Assign to nearest medoid. More robust to outliers than means; more computation.

GMM: SOFT CLUSTERS + EM

Input C; μ_c: N-vector mean, Σ_c: N×N covariance, π_c: weight. Full Σ allows tilted/correlated clusters. Still Gaussian; no noise label. Scale features (course).
$p(x) = \sum_c \pi_c \varphi_c(x)$; $\pi_c \geq 0$, $\sum_c \pi_c = 1$
$\varphi_c(x)$ = Gaussian density $N(x \mid \mu_c, \Sigma_c)$.
E-step: parameters → probabilities (Bayes). C responsibilities per point, summing to 1.
$\tau_{ic} = P(c \mid x_i) = \pi_c \varphi_c(x_i) / \sum_r \pi_r \varphi_r(x_i)$
M-step: probabilities → parameters (maximize expected complete-data log-likelihood). Soft count $n_c = \sum_i \tau_{ic}$.
$\mu_c = \sum_i \tau_{ic} x_i / n_c$; $\pi_c = n_c/M$
$\Sigma_c = \sum_i \tau_{ic}(x_i - \mu_c)(x_i - \mu_c)^T / n_c$
Use **new μ** in Σ; denominator n_c, not $n_c - 1$. Larger τ → more influence. **π_c = average membership over points**; τ_{ic} = one point's membership.
Initialize → E → M → E … until stable. E gets parameters from **previous M**; M uses latest E probabilities. Hard label = largest τ.
E ↔ Assignment; M ↔ Update. Practice: hard 0/1 probabilities + diagonal Σ ⇒ Lloyd is **true in course**. Exact Euclidean form: shared $\sigma^2 I$, equal weights (or small-variance limit).
BIC = −2 ln L + q ln M; $L = \prod_i p(x_i)$
Lower BIC wins. More clusters: likelihood↑ (term 1↓), parameter count q↑ (term 2↑). Balances fit vs complexity.
Generative: estimates source density; generate by choosing c via π, then sampling x from its Gaussian.

DB-SCAN + SILHOUETTE

DB-SCAN: nonparametric, arbitrary shapes, outliers; **no K input**. Radius ε (= tol); min_samples sets density (course: min cluster size). Grow via nearby points; isolated/sparse groups → noise. Scale.
tol too large: joins distinct clusters. **Too small:** fragments clusters/more noise. min_samples↑ rejects more sparse groups. Sensitive to radius; varying densities are hard → OPTICS/HDBSCAN (more computation).
Silhouette: $s = (b - a)/\max(a, b)$, in [−1, 1]
Slides: a = avg point-to-own-centroid distance; b = avg centroid-pair distance. 1 = separated; 0 = overlap; negative = poor assignment. Applicable across clustering models; inertia/elbow poor for DB-SCAN.
Standard: a = mean distance to own-cluster peers; b = nearest other cluster's mean distance; average s_i. Follow the question's convention.